

Manual de Execução, Testes Automatizados e Validação da Pipeline

Projeto: Flutter Test Architecture

Este documento apresenta o passo a passo para clonar, executar, testar e validar o projeto **Flutter Test Architecture**, incluindo a execução local da aplicação, testes automatizados, GitHub Actions, pipeline CI/CD e relatório HTML de cobertura de testes.

1. Clonando o Repositório do GitHub

1.1 Repositório do projeto

O projeto está disponível no GitHub no endereço:

<https://github.com/jhenriquedm/flutter-test-architecture>

1.2 Abrir o terminal

No Windows, abrir o PowerShell ou terminal do VS Code.

Escolher uma pasta local onde o projeto será salvo. Exemplo:

```
cd C:\Projeto-flutter
```

1.3 Clonar o projeto

Executar o comando:

```
git clone https://github.com/jhenriquedm/flutter-test-architecture.git
```

Esse comando baixa todo o código-fonte do projeto diretamente do GitHub para o computador local.

1.4 Entrar na pasta do projeto

Após o clone, acessar a pasta do projeto:

```
cd flutter-test-architecture
```

2. Requisitos para Executar a Aplicação

Para executar o projeto localmente, é necessário ter os seguintes recursos instalados:

2.1 Requisitos obrigatórios

- Flutter SDK;
 - Dart SDK;
 - Git;
 - VS Code ou Android Studio;
 - Navegador Chrome;
 - Extensões Flutter e Dart no VS Code.
-

2.2 Validar ambiente Flutter

Executar:

```
flutter doctor
```

Esse comando verifica se o ambiente está pronto para desenvolvimento Flutter.

Itens importantes:

```
[√] Flutter
[√] Android toolchain
[√] Chrome
[√] VS Code ou Android Studio
```

Caso o Visual Studio apareça como pendente, isso não impede a execução do projeto no Chrome ou Android, pois ele é necessário apenas para desenvolvimento Windows Desktop.

3. Instalando as Dependências do Projeto

Após clonar o projeto e entrar na pasta, executar:

```
flutter pub get
```

Esse comando baixa as dependências definidas no arquivo:

```
pubspec.yaml
```

Dependências principais utilizadas:

```
flutter  
cupertino_icons  
flutter_test  
flutter_lints
```

4. Abrindo e Executando a Aplicação

4.1 Abrir o projeto no VS Code

Na raiz do projeto, executar:

```
code .
```

4.2 Executar aplicação no Chrome

Executar:

```
flutter run -d chrome
```

Esse comando inicia a aplicação Flutter no navegador Chrome.

4.3 Executar em dispositivo disponível

Também é possível executar:

```
flutter run
```

Nesse caso, o Flutter perguntará ou escolherá um dispositivo disponível.

5. Credenciais para Login

A aplicação utiliza autenticação mockada, ou seja, os dados válidos estão definidos diretamente no código para fins de demonstração e testes.

Credenciais válidas

E-mail: admin@flutter.com

Senha: 12345678

6. Fluxo Funcional da Aplicação

O fluxo principal da aplicação é:

```
graph TD; Login --> Dashboard_QA1[Dashboard QA]; Dashboard_QA1 --> Test_Runner[Test Runner]; Test_Runner --> Dashboard_QA2[Dashboard QA]; Dashboard_QA2 --> Logout[Logout]; Logout --> Login;
```

7. Executando os Testes Locais

7.1 Comando para executar todos os testes

Na raiz do projeto, executar:

```
flutter test
```

Esse comando executa todos os testes automatizados presentes no diretório:

```
test/
```

7.2 Resultado esperado

Ao final da execução, o terminal deve exibir:

All tests passed!

Além disso, os testes exibem logs detalhados, permitindo acompanhar cada validação executada.

8. GitHub Actions

8.1 O que é GitHub Actions

O GitHub Actions é uma ferramenta de automação integrada ao GitHub. Ele permite executar tarefas automaticamente sempre que ocorre algum evento no repositório, como:

- push;
- pull request.

Neste projeto, ele é utilizado para:

- baixar o código;
 - instalar Flutter;
 - instalar dependências;
 - executar testes automatizados;
 - gerar coverage;
 - gerar relatório HTML;
 - publicar o relatório como artefato.
-

8.2 Arquivo da pipeline

A configuração da pipeline está no arquivo:

`.github/workflows/flutter-tests.yml`

8.3 Explicação da pipeline

A pipeline utilizada no projeto segue esta estrutura:

```
name: Flutter Automated Tests
```

Define o nome do workflow exibido na aba Actions do GitHub.

```
on:
  push:
    branches:
      - main
      - master

  pull_request:
    branches:
      - main
      - master
```

Define quando a pipeline será executada.

Neste caso, ela roda automaticamente quando há:

- push na branch main;
 - push na branch master;
 - pull request para main;
 - pull request para master.
-

```
jobs:
  test:
    name: Run Flutter Tests
    runs-on: ubuntu-latest
```

Define o job principal chamado Run Flutter Tests.

Ele será executado em uma máquina virtual Linux fornecida pelo GitHub:

ubuntu-latest

```
- name: Checkout do código
  uses: actions/checkout@v4
```

Essa etapa baixa o código do repositório para a máquina virtual do GitHub.

```
- name: Configurar Flutter
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.29.0'
    channel: 'stable'
```

Essa etapa instala e configura o Flutter na versão definida.

```
- name: Instalar dependências
  run: flutter pub get
```

Essa etapa instala as dependências do projeto.

```
- name: Executar testes automatizados
  run: flutter test --coverage
```

Essa etapa executa os testes automatizados e gera o arquivo de cobertura:

coverage/lcov.info

```
- name: Instalar LCOV
  run: sudo apt-get update && sudo apt-get install -y lcov
```

Essa etapa instala o LCOV na máquina virtual Linux.

O LCOV é necessário para converter o arquivo `lcov.info` em relatório HTML.

```
- name: Gerar relatório HTML de cobertura
  run: genhtml coverage/lcov.info -o coverage/html
```

Essa etapa gera o relatório HTML de cobertura.

O relatório é criado na pasta:

coverage/html

```
- name: Publicar relatório de cobertura como artefato
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: coverage/html
```

Essa etapa publica o relatório HTML como artefato do GitHub Actions.

O nome do artefato gerado é:

coverage-report

9. Como Executar a Pipeline no GitHub

A pipeline é executada automaticamente quando o código é enviado para o GitHub.

9.1 Verificar alterações locais

```
git status
```

9.2 Adicionar arquivos alterados

```
git add .
```

9.3 Criar commit

```
git commit -m "Descrição da alteração"
```

9.4 Enviar para o GitHub

```
git push
```

Após o push, o GitHub Actions será executado automaticamente.

10. Como Validar os Testes no GitHub Actions

10.1 Acessar o repositório

Abrir no navegador:

<https://github.com/jhenriquedm/flutter-test-architecture>

10.2 Abrir aba Actions

No GitHub, clicar na aba:

Actions

10.3 Selecionar workflow

Selecionar:

Add coverage report to GitHub Actions #3

10.4 Verificar status da execução

Os status possíveis são:

Verde: execução concluída com sucesso

Amarelo: execução em andamento

Vermelho: execução com falha

10.5 Validar logs da pipeline

Abrir a execução mais recente e verificar as etapas:

- Checkout do código;
- Configurar Flutter;
- Instalar dependências;
- Executar testes automatizados;
- Instalar LCOV;
- Gerar relatório HTML;
- Publicar artefato.

Se todas estiverem verdes, a pipeline foi executada corretamente.

← Flutter Automated Tests

✓ Add coverage report to GitHub Actions #3

Summary

All jobs

✓ Run Flutter Tests

Run details

Usage

Workflow file

Triggered via push last week

✓ jhenriquedm pushed -o 39d1523 main

flutter-tests.yml

on: push

✓ Run Flutter Tests

1m 15s

[Clicar aqui para acessar logs](#)

11. Como Baixar e Validar o Relatório HTML

11.1 Acessar execução correta

No GitHub, acessar:

Actions

Depois abrir a execução:

Add coverage report to GitHub Actions #3

Essa execução contém o relatório HTML gerado pela pipeline.

11.2 Baixar artefato

Na página da execução, rolar até o final e localizar a seção:

Artifacts

Baixar o arquivo:

coverage-report

O GitHub baixará um arquivo .zip.

11.3 Extrair relatório

Após baixar, extrair o arquivo .zip.

Dentro da pasta extraída, abrir:

index.html

Esse arquivo abrirá no navegador.

11.4 Validar informações do relatório

O relatório HTML apresenta:

- percentual geral de cobertura;
- cobertura por diretório;
- cobertura por arquivo;
- linhas cobertas;
- linhas não cobertas.

Resultado obtido no projeto:

Cobertura total: 96.8%

Screens: 95.8%

Widgets: 100%

Esse resultado indica que a maior parte do código testável da aplicação está coberta pelos testes automatizados.

12. Explicação do Código de Testes Local

Os testes estão localizados no arquivo:

test/widget/login_screen_test.dart

O arquivo utiliza os pacotes:

```
import 'package:flutter/material.dart';  
import 'package:flutter_test/flutter_test.dart';
```

E importa a tela de login:

```
import 'package:flutter_test_architecture/screens/login_screen.dart';
```

13. Estrutura Geral dos Testes

Todos os testes estão dentro da função:

```
void main() {  
  ...  
}
```

Cada cenário é implementado usando:

```
testWidgets(  
  'Descrição do teste',  
  (WidgetTester tester) async {  
    ...  
  },  
);
```

O WidgetTester permite:

- carregar telas;
- encontrar widgets;
- digitar textos;

- clicar em botões;
 - aguardar navegações;
 - validar resultados.
-

14. Cenário 1: Interface Inicial

Nome do teste

Validar se a interface inicial está sendo carregada corretamente.

Objetivo

Verificar se os principais elementos da tela de login foram renderizados corretamente.

Etapas executadas

1. Carregar tela de login

```
await tester.pumpWidget(  
  const MaterialApp(  
    home: LoginScreen(),  
  ),  
);
```

Essa etapa monta a tela de login em ambiente de teste.

2. Validar campo de e-mail

```
expect(find.byKey(const Key('email_input')), findsOneWidget);
```

Verifica se existe exatamente um campo com a chave:

email_input

3. Validar campo de senha

```
expect(find.byKey(const Key('password_input')), findsOneWidget);
```

Verifica se o campo de senha está presente.

4. Validar botão de login

```
expect(find.byKey(const Key('login_button')), findsOneWidget);
```

Verifica se o botão de login existe na tela.

5. Validar botão criar conta

```
expect(find.byKey(const Key('create_account_button')), findsOneWidget);
```

Verifica se o botão de criar conta foi renderizado.

Resultado esperado

Todos os elementos principais da tela de login devem existir.

15. Cenário 2: Login Inválido

Nome do teste

Garantir que o sistema trate credenciais inválidas.

Objetivo

Validar se o sistema bloqueia login com e-mail inválido e exibe mensagem de erro.

Etapas executadas

1. Carregar a tela de login

A tela é carregada usando pumpWidget.

2. Inserir e-mail inválido

```
await tester.enterText(  
  find.byKey(const Key('email_input')),  
  'emailinvalido',  
);
```

Esse e-mail não possui formato válido.

3. Inserir senha

```
await tester.enterText(  
  find.byKey(const Key('password_input')),  
  '12345678',  
);
```

4. Acionar botão ENTRAR

```
await tester.tap(  
  find.byKey(const Key('login_button')),  
);
```

5. Atualizar interface

```
await tester.pump();
```

Esse comando força a atualização visual da interface após o clique.

6. Validar mensagem de erro

```
expect(  
  find.text('Digite um e-mail válido'),  
  findsOneWidget,  
);
```

Verifica se a mensagem de erro foi exibida.

7. Validar que não navegou para o Dashboard

```
expect(  
  find.text('Dashboard QA'),  
  findsNothing,  
);
```

Garante que o usuário permaneceu na tela de login.

Resultado esperado

O sistema deve exibir erro e impedir a navegação.

16. Cenário 3: Login Válido

Nome do teste

Validar autenticação correta.

Objetivo

Validar se o login com credenciais corretas leva o usuário para o Dashboard.

Etapas executadas

1. Inserir e-mail válido

```
await tester.enterText(  
  find.byKey(const Key('email_input')),  
  'admin@flutter.com',  
);
```

2. Inserir senha válida

```
await tester.enterText(  
  find.byKey(const Key('password_input')),  
  '12345678',  
);
```

3. Clicar em ENTRAR

```
await tester.tap(  
  find.byKey(const Key('login_button')),  
);
```

4. Aguardar navegação

```
await tester.pumpAndSettle();
```

Esse comando aguarda animações e navegações finalizarem.

5. Validar Dashboard

```
expect(  
  find.text('Dashboard QA'),  
  findsOneWidget,  
);
```

6. Validar botão de logout

```
expect(  
  find.byKey(const Key('logout_button')),  
  findsOneWidget,  
);
```

Resultado esperado

O usuário deve ser autenticado e direcionado ao Dashboard.

17. Cenário 4: Logout

Nome do teste

Validar processo de logout e retorno para tela de login com campos limpos

Objetivo

Validar se o logout retorna para a tela de login e limpa os campos preenchidos.

Etapas executadas

1. Realizar login válido

O teste preenche e-mail e senha válidos, depois aciona o botão de login.

2. Validar Dashboard

```
expect(  
  find.text('Dashboard QA'),  
  findsOneWidget,  
);
```

3. Clicar no botão de logout

```
await tester.tap(  
  find.byKey(const Key('logout_button')),  
);
```

4. Aguardar retorno

```
await tester.pumpAndSettle();
```

5. Validar retorno ao login

```
expect(  
  find.text('Flutter Test Architecture'),  
  findsOneWidget,  
);
```

6. Validar limpeza do e-mail

```
expect(  
  find.text('admin@flutter.com'),  
  findsNothing,  
);
```

7. Validar limpeza da senha

```
expect(  
  find.text('12345678'),  
  findsNothing,  
);
```

Resultado esperado

O sistema deve retornar para o login e limpar os campos.

18. Cenário 5: Navegação entre Telas

Nome do teste

Validar navegação entre telas.

Objetivo

Validar o fluxo de navegação entre Dashboard e Test Runner.

Etapas executadas

1. Realizar login válido

O teste autentica o usuário com credenciais válidas.

2. Validar Dashboard

```
expect(  
  find.text('Dashboard QA'),  
  findsOneWidget,  
);
```

3. Clicar no botão Abrir Test Runner

```
await tester.tap(  
  find.byKey(const Key('open_test_runner_button')),  
);
```

4. Aguardar navegação

```
await tester.pumpAndSettle();
```

5. Validar tela Test Runner

```
expect(  
  find.text('Test Runner'),  
  findsOneWidget,  
);
```

6. Validar informações de cobertura

```
expect(  
  find.text('Cobertura Geral'),  
  findsOneWidget,  
);
```

7. Garantir visibilidade do botão voltar

```
await tester.ensureVisible(  
  find.byKey(const Key('back_home_button')),  
);
```

8. Clicar no botão voltar

```
await tester.tap(  
  find.byKey(const Key('back_home_button')),
```

```
warnIfMissed: false,  
);
```

9. Validar retorno ao Dashboard

```
expect(  
  find.text('Dashboard QA'),  
  findsOneWidget,  
);
```

Resultado esperado

O usuário deve navegar para Test Runner e retornar corretamente ao Dashboard.

19. Importância das Keys nos Testes

Os testes utilizam Key para localizar os widgets de forma precisa.

Exemplos:

```
email_input  
password_input  
login_button  
logout_button  
open_test_runner_button  
back_home_button
```

Essa abordagem é importante porque:

- reduz dependência de textos visíveis;
 - melhora a estabilidade dos testes;
 - facilita manutenção;
 - torna os testes mais confiáveis.
-

20. Considerações Finais

O projeto demonstra um fluxo completo de automação de testes em Flutter, contemplando:

- execução local dos testes;
- logs detalhados;
- validação de interface;
- validação de fluxo;

- GitHub Actions;
- coverage report;
- relatório HTML;
- pipeline CI/CD.

Esse conjunto permite apresentar uma solução prática, mensurável e profissional para automação de testes em aplicações mobile Flutter.